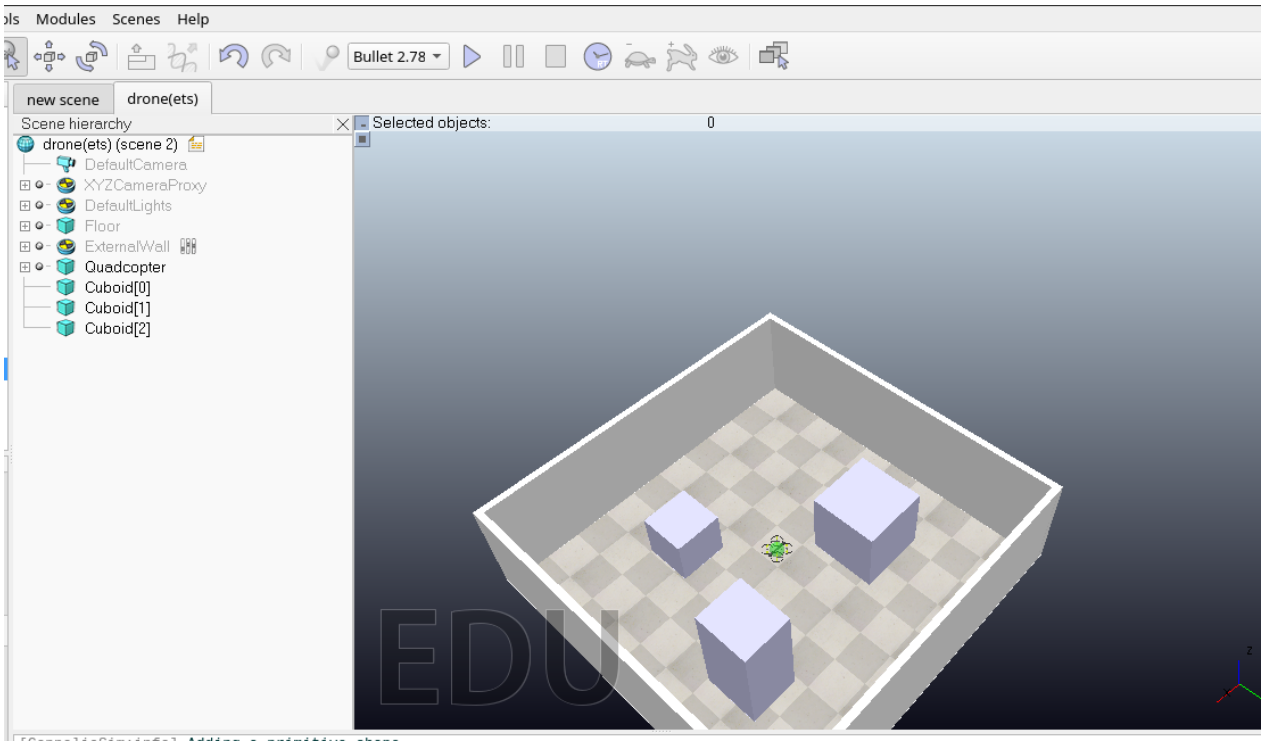
DOKUMENTASI TEKNIS PROYEK SIMULASI

PEMETAAN LINGKUNGAN 3D MENGGUNAKAN QUADCOPTER

CoppeliaSim · Fast 3D Laser Scanner · Python ZMQ · Voxel Cartesian



Gambar 1. Scene CoppeliaSim: Quadcopter, Floor, ExternalWall, dan tiga Cuboid. Sumber: tangkapan layar proyek.

Ruang lingkup

Sistem menggerakkan drone dengan FlightMission Lua, mengambil data kedalaman melalui Python ZMQ, membentuk point cloud 3D, dan memetakan voxel terukur. Keluaran akhir menggunakan tampilan Cartesian X/Y/Z numerik dengan pewarnaan berdasarkan elevasi Z.

Tahap awal	Versi akhir yang didokumentasikan
Point cloud dan pusat voxel pada kisi 8 cm; hasil PNG dan PLY.	Voxel 4 cm, bidang muka kubus teramati, sumbu X/Y/Z bernilai meter, colorbar Z dan lintasan biru.

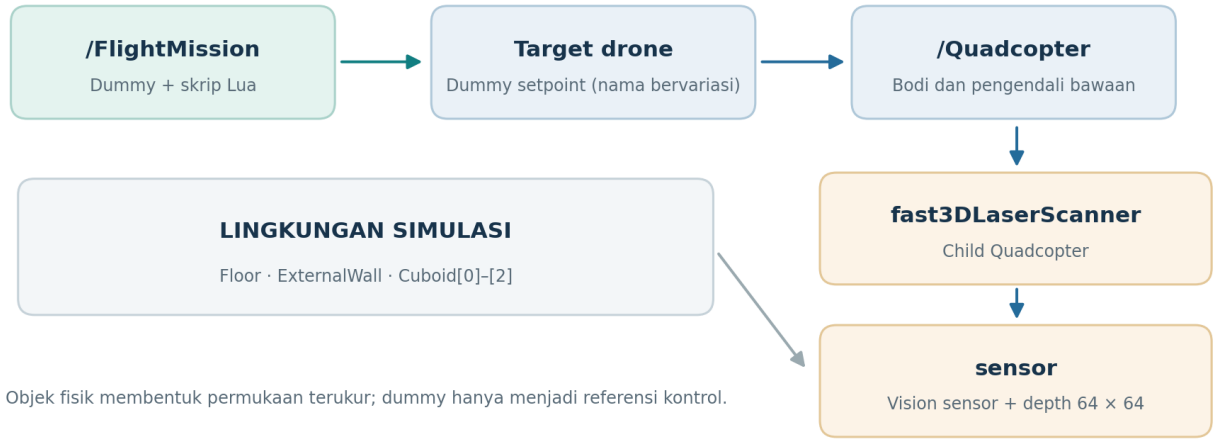
Dasar dokumen: main_uts_fixed.py (versi awal), main(2).py (referensi visual), dan main.py bersih sepanjang 399 baris (versi akhir).

Catatan dokumentasi: gambar Cartesian yang digunakan sebagai contoh berasal dari pengujian offline dengan data sintetis, bukan bukti akurasi pemindaian scene nyata.

1. Misi dan aset simulasi

Misi: menghasilkan representasi 3D lingkungan tertutup dari pembacaan depth scanner selama Quadcopter mengikuti waypoint. Python mencatat lintasan, mengubah titik lokal sensor menjadi koordinat dunia, lalu mengelompokkan titik ke voxel.

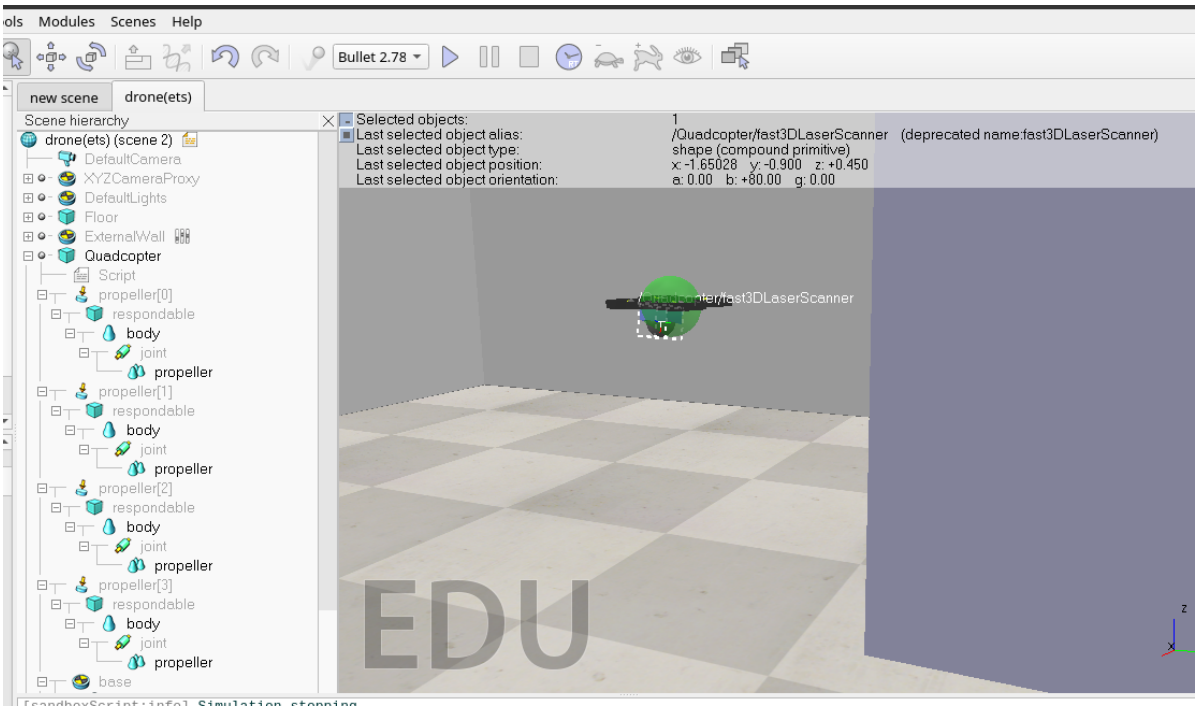
Komponen	Peran dalam misi
/Quadcopter	Bodi drone; posisi dan orientasi menjadi acuan transformasi dunia.
Target drone (dummy)	Setpoint posisi yang diikuti pengendali Quadcopter bawaan; alias dapat berbeda.
/FlightMission (dummy)	Menyimpan simulation script Lua untuk urutan waypoint dan status misi.
fast3DLaserScanner	Model pemindai yang menjadi child Quadcopter.
Vision sensor /sensor	Menyediakan depth buffer. Resolusi yang pernah terdiagnosis: 64 × 64.
Floor / ExternalWall	Permukaan lantai dan pembatas arena pemetaan.
Cuboid[0]–[2]	Tiga penghalang berbentuk kotak sebagai target pemindaian.



Gambar 2. Hierarki, relasi kontrol dummy, serta lingkungan yang dipindai.

Dummy bukan objek hasil ukur. /FlightMission mengatur target kendali; Python tidak memindahkan bodi drone secara langsung.

2. Kendali terbang dan pemeriksaan sensor



Gambar 3. fast3DLaserScanner dipasang pada Quadcopter; orientasi dan posisi mengikuti parent drone. Sumber: tangkapan layar proyek.

2.1 Urutan misi

Simulation script Lua pada /FlightMission tetap digunakan tanpa perubahan pada revisi akhir. Python hanya menyiapkan stepping, mengambil sampel sensor, membaca posisi drone, dan berhenti saat sinyal penyelesaian muncul atau batas waktu tercapai.

```
sim.setStepping(True)
sim.startSimulation()
# di dalam loop akuisisi:
sim.step()
pos = sim.getObjectPosition(drone, sim.handle_world)
```

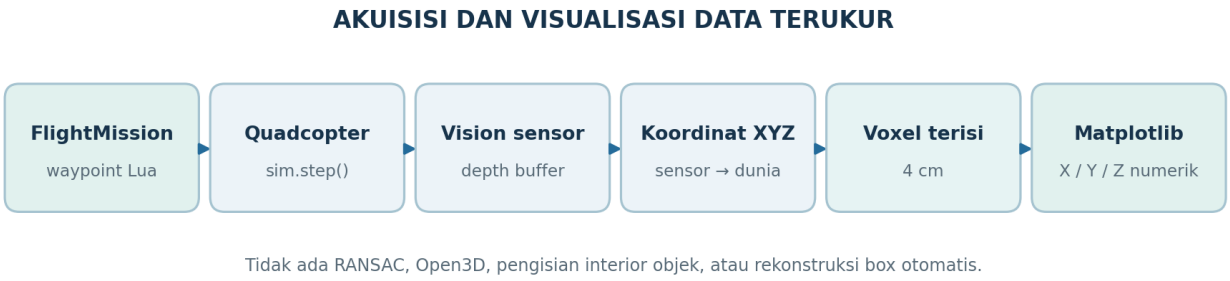
sim.setStepping(True) menyinkronkan langkah simulasi dengan Python. sim.step() memajukan simulasi satu langkah; getObjectPosition() merekam XYZ dunia untuk lintasan.

2.2 Pembacaan status waypoint

```
done = sim.getIntProperty(
    sim.handle_scene, 'signal.droneMappingDone', {'noError': True}
)
if done == 1:
    break
```

Sinyal scene droneMappingDone digunakan sebagai kondisi selesai; kode juga membatasi MAX_SIM_SECONDS = 165 s dan MAX_WALL_SECONDS = 300 s. Detail titik waypoint tetap mengikuti Lua yang sudah terpasang.

3. Depth, koordinat sensor, dan transformasi



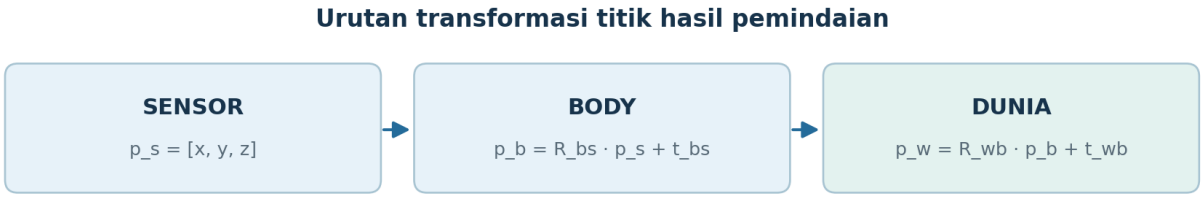
Gambar 4. Pipeline akhir; kendali dan pemindaian dipisahkan dari visualisasi.

3.1 Membaca depth

```
raw, resolution = sim.getVisionSensorDepth(info['handle'], 1)
local = depth_points(raw, resolution, info, args.pixel_stride)
```

Argumen 1 meminta depth dalam satuan meter. depth_points() mengubah piksel (u, v) dan jarak d menjadi koordinat 3D lokal sensor memakai proyeksi perspektif. Titik di luar near/far clipping dan jarak minimum 0,25 m dibuang.

3.2 Transformasi sensor → drone → dunia



Gambar 5. Komposisi transformasi posisi dan orientasi pada setiap frame scan.

```
T_body_sensor = matrix4(sim.getObjectMatrix(info['handle'], drone))
T_world_body  = matrix4(sim.getObjectMatrix(drone, sim.handle_world))
body  = local @ T_body_sensor[:3, :3].T + T_body_sensor[:3, 3]
world = body  @ T_world_body[:3, :3].T  + T_world_body[:3, 3]
```

matrix4() mengubah 12 nilai matriks objek menjadi matriks homogen 4 × 4. Perkalian rotasi dan translasi menghasilkan titik absolut P_world pada koordinat scene.

3.3 Data lintasan

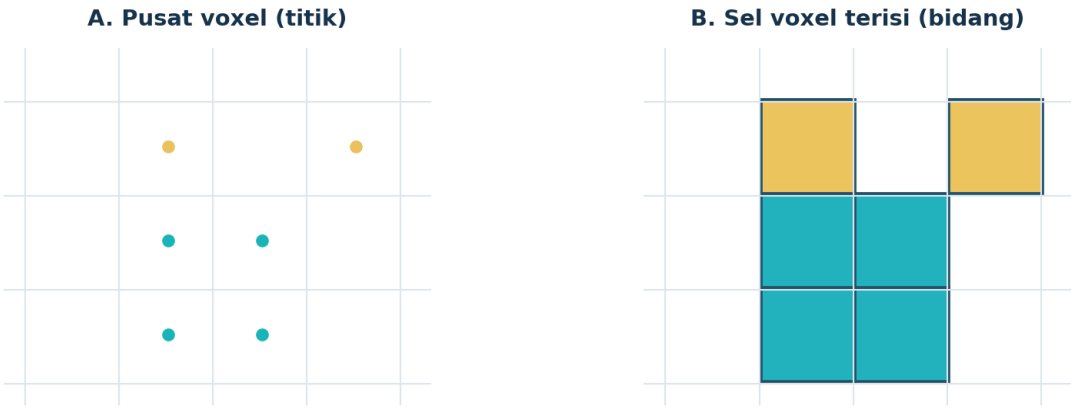
```
path.append([sim.getSimulationTime(), *pos])
```

Setiap baris lintasan memuat waktu simulasi, X, Y, dan Z. Berkas keluaran: drone_trajectory.csv.

4. Hasil awal dan peningkatan voxel

Implementasi awal menggunakan sampling depth setiap 6 piksel dan interval 0,45 s. Hasil direpresentasikan sebagai point cloud dan pusat voxel yang diberi warna menurut Z. Bentuk sisi Cuboid belum tampak kontinu ketika sampel terlalu jarang.

Parameter	Versi awal	Versi akhir
PIXEL_STRIDE	6	2
ACQUIRE_EVERY	0,45 s	0,20 s
VOXEL_SIZE	0,08 m	0,04 m
Tampilan voxel	Pusat sel / titik	Muka kubus sel terisi
Viewer 3D	PNG / scatter Matplotlib	Matplotlib Cartesian interaktif



Gambar 6. Skema representasi pusat sel dan bidang voxel terisi (ilustrasi, bukan data scene).

4.1 Konversi titik ke indeks voxel

```
keys = np.unique(
    np.floor(points / voxel_size).astype(np.int64), axis=0
)
centers = (keys + .5) * voxel_size
```

np.floor() mengubah koordinat XYZ menjadi indeks kisi tiga dimensi. np.unique() menyisakan satu entri untuk setiap sel yang terisi. Voxel kosong tidak diisi; interior objek tidak diasumsikan padat.

4.2 Batas densitas tampilan

Parameter --display-limit bernilai awal 4500 sel. Jika voxel terukur lebih banyak, ukuran sel untuk mesh tampilan dinaikkan bertahap. Data voxel dengan ukuran asli 4 cm tetap disimpan di CSV/PLY.

5. Pembuatan muka kubus dan warna ketinggian

5.1 Muka voxel yang terlihat

visible_quads(keys, size) membentuk enam kandidat sisi kubus per sel. Sisi yang berbatasan dengan sel terisi lain dilewati agar tidak merender bidang internal yang tersembunyi.

```
for key in keys:
    vertices = (corners + key) * size
    for offset, face in sides:
        if tuple(key + offset) not in occupied:
            quads.append(vertices[list(face)])
```

Hasil quads berupa kumpulan bidang segi empat pada lokasi voxel yang benar-benar memiliki titik hasil pemindaian.

5.2 Pewarnaan berdasarkan elevasi dunia

```
norm = colors.Normalize(zlo, zhi if zhi > zlo else zlo + .001)
cmap = plt.get_cmap('turbo')
face_colors = cmap(norm(face_z))
```

Warna menunjukkan koordinat Z dunia (m), bukan nilai tinggi fisik tiap Cuboid. Colorbar dan angka pada sumbu Z menggunakan data elevasi terukur.

5.3 Penggambaran sebagai bidang 3D

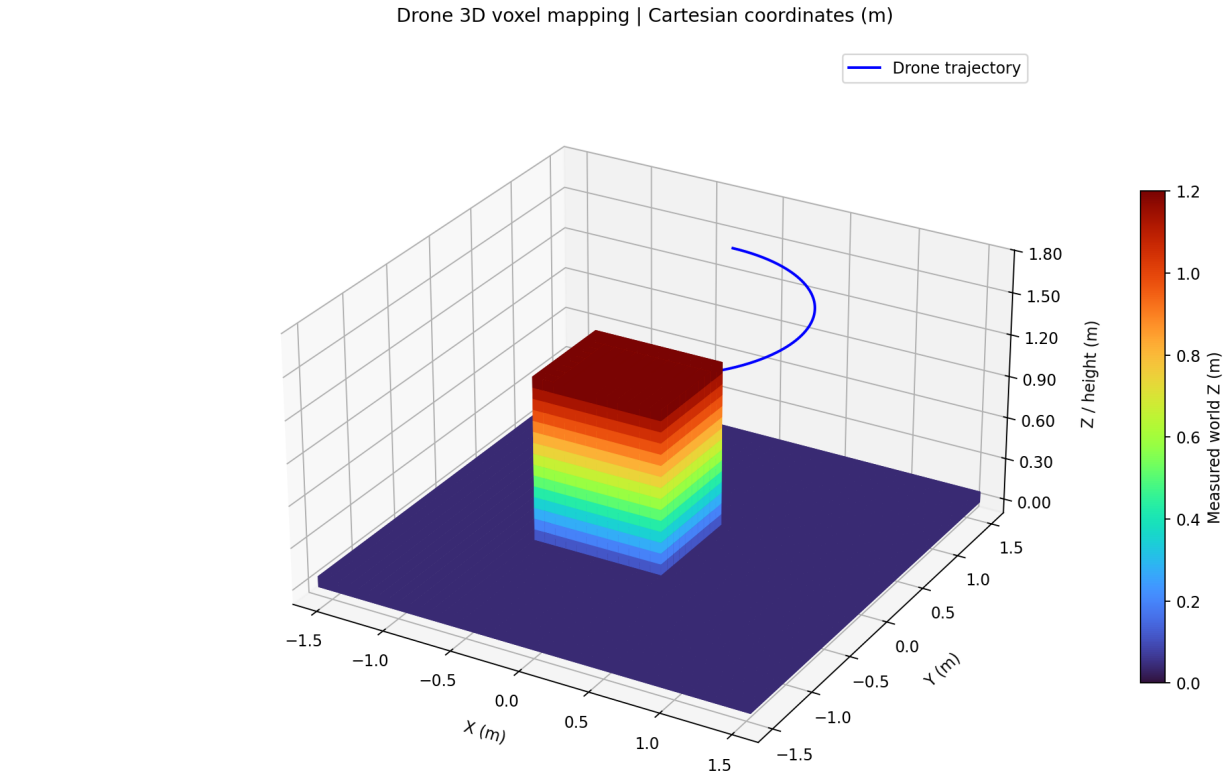
```
mesh = Poly3DCollection(
    quads, facecolors=face_colors,
    linewidths=0, edgecolors='none', zsort='average'
)
ax.add_collection3d(mesh)
```

Poly3DCollection menggambar muka kubus dalam satu plot 3D Matplotlib. Peta tidak membangun permukaan baru pada area tanpa data dan tidak melakukan fitting kotak.

5.4 Perbedaan dua pilihan visualisasi

Opsi CLI	Tampilan
--mode cubes (default)	Bidang permukaan voxel; bentuk terlihat lebih padat.
--mode points	Titik pusat voxel yang terukur; lebih ringan untuk meninjau sebaran.

6. Visualisasi akhir: Cartesian X/Y/Z



Gambar 7. Tampilan Cartesian versi akhir. Contoh uji offline dengan Cuboid sintetis; bukan hasil scan scene pengguna.

Tampilan akhir menggunakan grid Cartesian dan sumbu X, Y, Z dalam meter. Sumbu Z diberi angka dua desimal. Lintasan drone berupa garis biru; tinggi setiap voxel ditunjukkan oleh warna turbo dan colorbar Measured world Z (m).

```
ax.set_xlabel('X (m)')
ax.set_ylabel('Y (m)')
ax.set_zlabel('Z / height (m)')
ax.zaxis.set_major_locator(ticker.MaxNLocator(nbins=7))
ax.zaxis.set_major_formatter(ticker.FormatStrFormatter('%.2f'))
ax.plot(path[:, 1], path[:, 2], path[:, 3], color='blue')
```

FormatStrFormatter('%.2f') menghasilkan label, misalnya 0.00, 0.50, 1.00 m. Nilai sumbu mengikuti koordinat dunia hasil scan; batas Z bawah mencakup Z = 0 bila diperlukan.

7. Berkas keluaran, eksekusi, dan status pengujian

Berkas	Isi
mapping_data.npz	Titik XYZ dunia dan lintasan [waktu, X, Y, Z].
drone_trajectory.csv	Lintasan Quadcopter pada setiap langkah simulasi.
raw_scan.ply	Point cloud mentah berwarna merah.
voxel_xyz.csv	Koordinat pusat voxel terukur, resolusi dasar 4 cm.
voxel_height_coloured.ply	Pusat voxel dengan warna berdasarkan Z.
01_mapping_2d.png	Proyeksi XY dan lintasan drone.
03_voxel_cartesian_3d.png	Plot tiga dimensi dengan sumbu X/Y/Z numerik.
04_display_voxel_cubes.ply	Bidang kubus yang dipakai pada tampilan; resolusi dapat adaptif.

7.1 Perintah program

```
python main.py
python main.py --view results/mapping_data.npz
python main.py --view results/mapping_data.npz --mode points
python main.py --view results/mapping_data.npz --headless
```

Eksekusi pertama membutuhkan CoppeliaSim dalam keadaan scene terbuka dan simulasi berhenti. Parameter --view memproses ulang scan tersimpan tanpa menerbangkan drone. --headless membuat ekspor tanpa jendela interaktif.

7.2 Status dan batas interpretasi

Gambar 7 adalah keluaran uji sintetis offline; laporan ini tidak menyajikan angka akurasi geometri Cuboid dari scene pengguna.

Sel terisi menunjukkan ruang yang tersentuh sampel depth, bukan bukti bahwa seluruh permukaan objek telah terukur. Warna Z menunjukkan elevasi absolut, bukan tinggi objek yang telah dipisahkan dari lantai. Penilaian galat dimensi memerlukan pembandingan dengan ukuran objek pada CoppeliaSim.